

Tutorial and Exercise

**T-SQL: Semi-Structured Data Handeling
(JSON)**

Author: Dr. Maqbool Hussain

University of Derby

Extending the DEPARTMENT Table Schema & Working with JSON Data

Objectives: Processing JSON in a Relational Database using T-SQL

1. **Store JSON Data in SQL Server**
 - Use `NVARCHAR(MAX)` to store JSON data in a relational table.
 - Ensure proper schema design for hybrid relational-JSON storage.
2. **Insert JSON Data into a Table**
 - Use `INSERT INTO` statements with JSON-formatted strings.
 - Ensure proper JSON syntax to avoid errors.
3. **Query JSON Data Efficiently**
 - Use `JSON_VALUE()` to extract **scalar values** (e.g., budget, location details).
 - Use `JSON_QUERY()` to extract **arrays or objects** (e.g., contacts, nested locations).
4. **Convert JSON Arrays into Rows**
 - Use `OPENJSON()` to **split JSON arrays into multiple rows**.
 - Use `CROSS APPLY` to join extracted JSON rows with the main table.
5. **Filter and Search JSON Data**
 - Use `WHERE JSON_VALUE(column, '$.key') = 'value'` for **filtering JSON fields**.
6. **Format Query Results into Tabular Structure**
 - Extract **nested JSON values** into separate columns.
 - Convert JSON **arrays** (e.g., contacts) into **individual rows** for better readability.

Steps to Implement

Step 1: Open Query Editor from SQL Server Managment Studio

- Launch your SQL query editor by clicking **New Query** window.

Step 2: Select the Database

- Switch to the **Test** database, which contains the required product information.

USE Test;

Step 1: Alter the DEPARTMENT Table to Add the dept_details Column

We will extend the `[dbo].[DEPARTMENT]` table by adding a new column `dept_details` of type `NVARCHAR(MAX)`, which will store JSON data (sample shown).

```
-- Adding column dept_details column to Department table
/*    dept_details column include JSON data with following key details.
    {
        "dept_budget": 500000,
        "dept_contacts": [
            "+44-555-1234",
            "+44-555-5678"
        ],
        "dept_location": {
            "building": "Main Office",
            "floor": 3,
            "room": "311"
        }
    }
*/

IF NOT EXISTS (
    SELECT 1 FROM INFORMATION_SCHEMA.COLUMNS
    WHERE TABLE_NAME = 'Department'
    AND COLUMN_NAME = 'dept_details'
)
BEGIN
    ALTER TABLE Department
    ADD dept_details NVARCHAR(MAX) NULL;
END
```

Step 2: Inspect the tables - to see the data and structure.

```
-- Inspecting the data in Department table
SELECT * FROM DEPARTMENT;
```

Step 3: Insert a New Row with JSON Data

Now, let's insert a new department along with the required JSON format in the `dept_details` column.

```
-- Adding new department Marketing, with corresponding budget and other
details in JSON
INSERT INTO [dbo].[DEPARTMENT] (NAME, dept_details)
VALUES
(
    'Marketing',
    '{
        "dept_budget": 500000,
        "dept_contacts": [
            "+44-555-1234",
            "+44-555-5678"
        ],
        "dept_location": {
            "building": "Main Office",
            "floor": 3,
            "room": "311"
        }
    }'
);
```

Step 4: Inspect again the table; Department with id: 7 is added successfully.

```
-- Inspecting the data in Department table
SELECT * FROM DEPARTMENT;
```

	DEPT_ID	NAME	dept_details
1	1	Operations	NULL
2	2	Loans	NULL
3	3	Administration	NULL
4	4	IT	NULL
5	5	Finance	NULL
6	6	Research	NULL
7	7	Marketing	{ "dept_budget": 500000, "dept_contacts": ["+44-555-1234", "+44-555-5678"], "dept_location": { "building": "Main Office", "floor": 3, "room": "311" } }

Step 5: Extract JSON Data in a Table Format

To extract specific values from the JSON stored in `dept_details`, we will use SQL Server's built-in JSON functions:

```
--Extracting data for new record added
SELECT
    DEPT_ID,
    NAME,
    JSON_VALUE(dept_details, '$.dept_budget') AS dept_budget,
    JSON_QUERY(dept_details, '$.dept_contacts') AS dept_contacts,
    JSON_VALUE(dept_details, '$.dept_location.building') AS building,
    JSON_VALUE(dept_details, '$.dept_location.floor') AS floor,
    JSON_VALUE(dept_details, '$.dept_location.room') AS room
FROM [dbo].[DEPARTMENT]
WHERE NAME = 'Marketing';
```

Use of JSON functions and key outcomes:

- `JSON_VALUE()` : Extracts a single scalar value from JSON.
- `JSON_QUERY()` : Extracts an array or object. (e.g. `dept_details`)

	DEPT_ID	NAME	dept_budget	dept_contacts	building	floor	room
1	7	Marketing	500000	["+44-555-1234", "+44-555-5678"]	Main Office	3	311

Step 5: Extract and Display `dept_contacts` in a Tabular Format

In **Step 4**, the `dept_contacts` field was displayed as a JSON array. To properly format it into multiple rows, we will use `OPENJSON()` to split the array into separate rows.

```
-- Extracting data for new record added - (nested array processed)
SELECT
    d.DEPT_ID,
    d.NAME,
    JSON_VALUE(d.dept_details, '$.dept_budget') AS dept_budget,
    JSON_VALUE(d.dept_details, '$.dept_location.building') AS building,
    JSON_VALUE(d.dept_details, '$.dept_location.floor') AS floor,
    JSON_VALUE(d.dept_details, '$.dept_location.room') AS room,
    c.contact_number
FROM [dbo].[DEPARTMENT] d
    CROSS APPLY OPENJSON(d.dept_details, '$.dept_contacts')
    WITH (contact_number NVARCHAR(50) '$') AS c
```

```
WHERE d.NAME = 'Marketing';
```

Use of JSON functions and key outcomes:

- **OPENJSON(dept_details, '\$.dept_contacts')**: Splits the JSON array into multiple rows.
- **CROSS APPLY**: Allows joining the exploded rows with the main department data.
- **WITH (contact_number NVARCHAR(50) '\$')**: Defines how we extract the values from the JSON array. Here each array element become on row of table which is aliased as 'c'.

	DEPT_ID	NAME	dept_budget	building	floor	room	contact_number
1	7	Marketing	500000	Main Office	3	311	+44-555-1234
2	7	Marketing	500000	Main Office	3	311	+44-555-5678

END of Tutorial

Exercises: Working with JSON Data in SQL Server

Exercise 1: Updating JSON Data for an Existing Department

Scenario:

You need to update the `dept_details` JSON column for the department named **"Finance"**. Update its budget and location details while ensuring previous data is replaced correctly. After update, use `SELECT` to ensure the data is updated successfully.

Task: Update the department where `NAME = 'Finance'` with the following JSON structure:

```
{
  "dept_budget": 750000,
  "dept_contacts": [
    "+44-555-9999",
    "+44-555-8888"
  ],
  "dept_location": {
    "building": "Finance Tower",
    "floor": 10,
    "room": "1005"
  }
}
```

Expected Outcome:

(1 row affected)

(1 row affected)

Completion time: 2025-03-03T13:04:45.3034953+00:00

	DEPT_ID	NAME	dept_details
1	5	Finance	{ "dept_budget": 750000, "dept_contacts": ["+44-555-9999", "+44-555-8888"], "dept_location": { "building": "Finance Tower", "floor": 10, "room": "1005" } }

Exercise 2: Insert a New Department for Industrial Spinout Ideas

Scenario:

A new department is being introduced to convert **ideas into industrial spinouts**. Insert this department into the database with the following JSON details: (Also, verify the department is successfully added using SELECT.)

Department Name: **Innovation Hub**

JSON Details:

```
{
  "dept_budget": 1200000,
  "dept_contacts": [
    "+44-555-7777",
    "+44-555-6666",
    "+44-555-2233"
  ],
  "dept_location": {
    "building": "Innovation Center",
    "floor": 5,
    "room": "502"
  }
}
```

Expected Outcome:

If successfully added; the SELECT will result in following.

	DEPT_ID	NAME	dept_details
1	9	Innovation Hub	{ "dept_budget": 1200000, "dept_contacts": ["+44-555-7777", "+44-555-6666", "+44-555-2233"], "dept_location": { "building": "Innovation Center", "floor": 5, "room": "502" } }

Exercise 3: Display All Departments with Flattened JSON Details

Scenario:

You need to retrieve all departments **that have dept_details available** and display them in a structured tabular format by extracting JSON fields. The results should include:

- Department **ID** and **Name**
- **Budget** (from JSON)
- **Contacts** (each contact displayed in a separate row)
- **Building, Floor, Room** (from JSON)

Expected Outcome:

	DEPT_ID	NAME	dept_budget	building	floor	room	contact_number
1	5	Finance	750000	Finance Tower	10	1005	+44-555-9999
2	5	Finance	750000	Finance Tower	10	1005	+44-555-8888
3	7	Marketing	500000	Main Office	3	311	+44-555-1234
4	7	Marketing	500000	Main Office	3	311	+44-555-5678
5	9	Innovation Hub	1200000	Innovation Center	5	502	+44-555-7777
6	9	Innovation Hub	1200000	Innovation Center	5	502	+44-555-6666
7	9	Innovation Hub	1200000	Innovation Center	5	502	+44-555-2233

Exercise 4: Modify the Building for 'Innovation Hub' to 'Finance Tower'

Scenario:

The **Innovation Hub** department has relocated from **Innovation Center** to **Finance Tower**. You need to update the `dept_details` JSON field to reflect this change while keeping all other details intact.

Task: Update the Building Name in JSON

Modify the JSON field for `dept_details` only for the 'Innovation Hub' department, changing:

- `"building": "Innovation Center"`
To
- `"building": "Finance Tower"`

Use query from Exercise 3 to verify the department location is changed.

Expected Outcome:

	DEPT_ID	NAME	dept_budget	building	floor	room	contact_number
1	5	Finance	750000	Finance Tower	10	1005	+44-555-9999
2	5	Finance	750000	Finance Tower	10	1005	+44-555-8888
3	7	Marketing	500000	Main Office	3	311	+44-555-1234
4	7	Marketing	500000	Main Office	3	311	+44-555-5678
5	9	Innovation Hub	1200000	Finance Tower	5	502	+44-555-7777
6	9	Innovation Hub	1200000	Finance Tower	5	502	+44-555-6666
7	9	Innovation Hub	1200000	Finance Tower	5	502	+44-555-2233

Exercise 5: Extract All Departments Located in 'Finance Tower' with Flattened JSON Data

Scenario:

You need to retrieve all departments **whose building location is set to 'Finance Tower'** and display their details in a structured tabular format. The extracted details should include:

- **Department ID and Name**
- **Budget**
- **Contacts** (each contact should be displayed in a separate row)
- **Building, Floor, and Room**

Expected Outcome:

	DEPT_ID	NAME	dept_budget	building	floor	room	contact_number
1	5	Finance	750000	Finance Tower	10	1005	+44-555-9999
2	5	Finance	750000	Finance Tower	10	1005	+44-555-8888
3	9	Innovation Hub	1200000	Finance Tower	5	502	+44-555-7777
4	9	Innovation Hub	1200000	Finance Tower	5	502	+44-555-6666
5	9	Innovation Hub	1200000	Finance Tower	5	502	+44-555-2233

Exercise 6(Bonus with Solution): Display Only Departments That Share the Same Building Location

Scenario:

You need to retrieve **only the departments that share the same building location with at least one other department**. Departments that are the only ones in a building should be excluded from the results.

NOTE: Solution is provided, try yourself first and consult solution later.

Expected Outcome:

	DEPT_ID	NAME	dept_budget	building	floor	room	contact_number
1	5	Finance	750000	Finance Tower	10	1005	+44-555-9999
2	5	Finance	750000	Finance Tower	10	1005	+44-555-8888
3	9	Innovation Hub	1200000	Finance Tower	5	502	+44-555-7777
4	9	Innovation Hub	1200000	Finance Tower	5	502	+44-555-6666
5	9	Innovation Hub	1200000	Finance Tower	5	502	+44-555-2233

One possible Solution:

```
WITH BuildingCounts AS (  
    SELECT  
        JSON_VALUE(dept_details, '$.dept_location.building') AS building,  
        COUNT(*) AS department_count  
    FROM [dbo].[DEPARTMENT]  
    WHERE JSON_VALUE(dept_details, '$.dept_location.building') IS NOT NULL  
    GROUP BY JSON_VALUE(dept_details, '$.dept_location.building')  
    HAVING COUNT(*) > 1 -- Only include buildings with more than one department  
)  
SELECT  
    d.DEPT_ID,  
    d.NAME,  
    JSON_VALUE(d.dept_details, '$.dept_budget') AS dept_budget,  
    JSON_VALUE(d.dept_details, '$.dept_location.building') AS building,  
    JSON_VALUE(d.dept_details, '$.dept_location.floor') AS floor,  
    JSON_VALUE(d.dept_details, '$.dept_location.room') AS room,  
    c.contact_number  
FROM [dbo].[DEPARTMENT] d  
CROSS APPLY OPENJSON(d.dept_details, '$.dept_contacts')  
WITH (contact_number NVARCHAR(50) '$') c  
WHERE JSON_VALUE(d.dept_details, '$.dept_location.building') IN (  
    SELECT building FROM BuildingCounts -- Only include buildings with multiple  
    departments  
)  
ORDER BY building, floor, room;
```